Miscellaneous Problems of Time Complexity

Last Updated : 23 Jul, 2025

Prerequisite : Asymptotic Notations

Time Complexity :
Time complexity is the time needed by an algorithm expressed as a function of the size of a problem. It can also be defined as the amount of computer time it needs to run a program to completion. When we solve a problem of time complexity then this definition help the most -
"It is the number of operations an algorithm performs to complete its task with respect to the input size."

There are following some miscellaneous problems of time complexity which are always frequently asking in different types of quizzes.

1. What is the time complexity of the following code -

C++ C Java Python3 C# JavaScript

```
void function(int n)
{
    int i = 1, s = 1;
    while (s < n) {
        s = s + i;
        i++;
    }
}

// This code is contributed by Shubham Singh
```

Solution -
Time complexity = O(√n).

Explanation -
We can define the 'S' terms according to the relation $S_i = S_{i-1} + i$. Let *k* is the total number of iterations taken by the program

i	S
1	1
2	2
3	2 + 2
4	2 + 2 + 3
...	...
k	2 + 2 + 3 + 4 + ...+ k

When S>=n , then loop will stop at k^{th} iterations,

⇒ S>=n ⇒ S=n

⇒ 2 + 2 + 3 + 4 + ...+ k = n

⇒ 1 + (k * (k + 1))/2 = n

⇒ $k^2 = n$

k = √n

Hence, the time complexity is O(√n).

2. What is the time complexity of the following code :

C++ C Java Python3 C# JavaScript

```
void fun(int n)
{
    if (n < 5)
        cout << "GeeksforGeeks";
    else {
        for (int i = 0; i < n; i++) {
            cout << i;
        }
```

```
    }
}

// This code is contributed by Shubham Singh
```

Solution -

Time complexity = O(1) in best case and O(n) in worst case.

Explanation -

This program contains if and else condition. Hence, there are 2 possibilities of time complexity. If the value of n is less than 5, then we get only *GeeksforGeeks* as output and its time complexity will be O(1).

But, if n>=5, then for loop will execute and time complexity becomes O(n), it is considered as worst case because it takes more time.

3. What is the time complexity of the following code :

C++ C Java Python3 C# JavaScript

```
void fun(int a, int b)
{
    // Consider a and b both are positive integers
    while (a != b) {
        if (a > b)
            a = a - b;
        else
            b = b - a;
    }
}

// This code is contributed by Shubham Singh
```

Solution -

Time complexity = O(1) in best case and O(max(a, b)) worst case.

Explanation -

If the value of a and b are the same, then while loop will not be executed. Hence, time complexity will be O(1).

But if a!=b, then the while loop will be executed. Let a=16 and b=5;

no. of iterations	a	b
1	16	5
2	16-5=11	5
3	11-5=6	5
4	6-5=1	5
5	1	5-1=4
6	1	4-1=3
7	1	3-1=2
8	1	2-1=1

For this case, while loop executed 8 times (a/2⇒16/2⇒8).
If a=5 and b=16, then also the loop will be executed 8 times. So we can say that time complexity is O(max(a/2,b/2))⇒O(max(a, b)), it is considered as worst case because it takes more time.

4. What is the time complexity of the following code :

C++ C Java Python3 C# JavaScript

```
void fun(int n)
{
  for(int i=0;i*i<n;i++)
    cout<<"GeeksforGeeks";
}
```

```
// This code is contributed by Shubham Singh
```

Solution -

Time complexity = O(√n).

Explanation -

Let **k** be the no. of iteration of the loop.

i	i*i
1	1
2	2 2
3	3 2
4	4 2
...	...
k	k 2

⇒ The loop will stop when i * i >=n i.e., i*i=n

⇒ i*i=n ⇒ $k^2 = n$

⇒k =√n

Hence, the time complexity is O(√n).

5. What is the time complexity of the following code :

C++ C Java Python3 C# JavaScript

```
void fun(int n, int x)
{
    for (int i = 1; i < n; i = i * x) //or for(int i = n; i >=1; i
= i / x)
        cout << "GeeksforGeeks";
}

//This code is contributed by Shubham Singh
```

Solution -

Time complexity = $O(\log_x n)$.

Explanation -

Let **k** be the no. of iteration of the loop.

no. of itr	i=i*x
1	1*x=x
2	x*x=x 2
3	x 2 *x=x 3
...	...
k	x k-1 *x= x k

⇒ The loop will stop when i>=n ⇒ $x^k = n$

⇒ $x^k=n$ (Take log both sides)

⇒ $k=\log_x n$

⇒Hence, time complexity is $O(\log_x n)$.

6. What is the time complexity of the following code :

C++ C Java Python3 C# JavaScript

```
#include <iostream>
using namespace std;

void fun(int n)
{
    for (int i = 0; i < n / 2; i++)
        for (int j = 1; j + n / 2 <= n; j++)
            for (int k = 1; k <= n; k = k * 2)
                cout << "GeeksforGeeks";
}

int main()
{
  int n=8;
  fun(3);
}

// The code is contributed by Nidhi goel.
```

Solution -

Time complexity = $O(n^2\log_2 n)$.

Explanation -

Time complexity of 1st for loop = $O(n/2) \Rightarrow O(n)$.

Time complexity of 2nd for loop = $O(n/2) \Rightarrow O(n)$.

Time complexity of 3rd for loop = $O(\log_2 n)$. (refer question number - 5)

Hence, the time complexity of function will become $O(n^2\log_2 n)$.

7. What is the time complexity of the following code :

C++ C Java Python3 C# JavaScript

```
void fun(int n)
{
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j = j + i)
            cout << "GeeksforGeeks";
}


//This Code is contributed by Shubham Singh
```

Solution - Time complexity = O(nlogn).

Explanation -

Time complexity of 1st for loop = O(n). 2nd loop executes n/i times for each value of i.

Its time complexity is O($\sum^{n}_{i=1}$ n/i) ⇒ O(logn).

Hence, time complexity of function = O(nlogn).

8. What is the time complexity of the following code :

C++ C Java Python3 C# JavaScript

```
void fun(int n)
{
    for (int i = 0; i <= n / 3; i++)
        for (int j = 1; j <= n; j = j + 4)
            cout << "GeeksforGeeks";
}

// The code is contributed by Arushi Jindal.
```

Solution - Time complexity = O(n^2).

Explanation -

Time complexity of 1st for loop = O(n/3) ⇒ O(n).

Time complexity of 2nd for loop = O(n/4) ⇒ O(n).

Hence, the time complexity of function will become O(n^2).

9. What is the time complexity of the following code :

C++ C Java Python3 C# JavaScript

```
void fun(int n)
{
    int i = 1;
    while (i < n) {
        int j = n;
        while (j > 0) {
            j = j / 2;
        }
        i = i * 2;
    }
}

//This code is contributed by Shubham Singh
```

Solution - Time complexity = $O(\log^2 n)$.

Explanation -

In each iteration , i become twice (T.C=O(logn)) and j become half (T.C=O(logn)).

So, time complexity will become $O(\log^2 n)$.

We can convert this while loop into for loop.

for(int i = 1; i < n; i = i * 2)

for(int j=n ; j > 0; j = j / 2).

Time complexity of above for loop is also $O(\log^2 n)$.

10. Consider the following code, what is the number of comparisons made in the execution of the loop ?

C++ C Java Python3 C# JavaScript

```
void fun(int n)
{
    int j = 1;
    while (j <= n) {
        j = j * 2;
    }
}
```

```
// The code is contributed by Arushi Jindal.
```

Solution -

ceil($\log_2 n$)+1.

Explanation -

Let **k** be the no. of iteration of the loop. After the kth step, the value of j is 2^k. Hence, k=$\log_2 n$. Here, we use *ceil of* $\log_2 n$, because $\log_2 n$ may be in decimal. Since we are doing one more comparison for exiting from the loop.

So, the answer is ceil($\log_2 n$)+1.